

LLM Notes

2026 Spring

Day 12: CS336 Systems: Benchmarking

GPU 性能测量与 Warmup

主讲: Aaron

Fudan University

今日摘要

2026.7.14 完成 CS336 Assignment 2 的基础 Benchmark 脚本: 支持不同模型规模、前向传播、前向加反向传播和完整训练步, 实验表明, 反向传播耗时约为前向传播的两倍; GPU 测量前进行 warmup 能够显著降低一次性初始化开销造成的偏差和方差。

12.1 Benchmark 脚本设计

12.1.1 测量对象

基础 Benchmark 包含三种模式:

1. **Forward**: 仅执行模型前向传播;
2. **Forward + Backward**: 计算 logits、loss, 并执行反向传播;
3. **Train Step**: 在前向和反向传播后继续执行 AdamW 参数更新。

模型输入使用随机生成的 token ID, 统一采用:

$$B = 4, \quad S = 512, \quad V = 10,000.$$

每组标准实验先执行 5 次 warmup, 再进行 10 次正式测量, 统计运行时间的均值与标准差。

定义

定义 12.1.1 (GPU Benchmark). GPU Benchmark 是在固定模型、输入形状和运行环境下, 对目标计算过程进行重复计时。由于 CUDA 默认异步执行, 计时边界处需要调用 `torch.cuda.synchronize()`, 确保 CPU 等待 GPU 完成已经提

交的计算。

分析

单次 Benchmark 脚本只负责运行一个

(model size, mode, warmup steps)

组合，并将结果以 JSON 输出。外层调度脚本对配置文件中各实验维度取笛卡尔积，为每个组合启动独立进程，并将结果逐行写入 CSV。

独立进程能够减少不同实验之间的 CUDA 缓存、显存碎片和异常状态干扰；逐行写入则可以避免程序中途退出时丢失已经完成的结果。

12.1.2 Forward 模式中的梯度记录

纯前向测量使用：

`torch.inference_mode()`。

禁用 autograd 后, PyTorch 不再为反向传播保存计算图和中间激活。该修改对 Small 至 XL 模型的运行时间影响很小，但显著降低了显存占用，使 10B 模型能够完成纯前向传播。

备注

纯 Forward 测量和训练阶段的 Forward 并不完全等价：前者关闭了 autograd，而后者需要记录计算图。因此，使用不同模式的耗时之差估算 Backward 和 AdamW 开销时，只能将结果视为近似值。

12.2 不同模型规模的测量结果

12.2.1 原始结果与阶段耗时

反向传播和优化器耗时近似计算为：

$$T_{\text{backward}} \approx T_{\text{forward+backward}} - T_{\text{forward}},$$

$$T_{\text{optimizer}} \approx T_{\text{train step}} - T_{\text{forward+backward}}.$$

模型	Forward	Backward	AdamW	Train Step
Small	20.06 ms	41.31 ms	7.72 ms	69.09 ms
Medium	55.11 ms	113.52 ms	24.89 ms	193.52 ms
Large	123.68 ms	259.20 ms	52.02 ms	434.89 ms
XL	334.58 ms	732.38 ms	165.24 ms	1232.20 ms
10B	1286.29 ms	OOM	OOM	OOM

分析

Small 至 XL 模型中，反向传播与前向传播的耗时比分别约为：

$$2.06, \quad 2.06, \quad 2.10, \quad 2.19.$$

这与理论分析一致。对于线性变换

$$Y = XW,$$

前向传播主要计算一次矩阵乘法 XW ，而反向传播需要计算：

$$\nabla_X L = \nabla_Y L W^\top, \quad \nabla_W L = X^\top \nabla_Y L.$$

因此，反向传播包含两次主要矩阵乘法，计算量通常约为前向传播的两倍。随着模型规模增大，AdamW 更新全部参数所需的时间也明显增加。

12.2.2 10B 模型的显存限制

FP32 中每个参数占用 4 bytes，因此 10B 模型仅参数就约占：

$$10^{10} \times 4 \text{ bytes} = 40 \text{ GB}.$$

训练时还需要保存梯度、前向激活以及 AdamW 的一阶和二阶动量。仅参数、梯度和两份优化器状态就约为：

$$40 + 40 + 40 + 40 = 160 \text{ GB},$$

尚未计入激活。因此，10B 模型能够在 80GB GPU 上完成关闭梯度记录的前向传播，但无法完成 FP32 反向传播和完整训练步。

12.3 Warmup 实验

12.3.1 不同预热次数的结果

固定 Large 模型，分别使用 0, 1, 2, 5 次 warmup:

Warmup	Forward	Forward + Backward	Train Step
0	143.20 $\pm$ 60.53	413.65 $\pm$ 105.99	470.79 $\pm$ 104.10
1	123.42 $\pm$ 0.04	378.32 $\pm$ 18.29	434.63 $\pm$ 0.21
2	123.02 $\pm$ 0.03	385.53 $\pm$ 0.09	436.47 $\pm$ 0.11
5	124.05 $\pm$ 0.03	386.56 $\pm$ 0.17	438.37 $\pm$ 0.36

表中时间单位均为 ms，形式为：

mean $\pm$ standard deviation.

分析

不进行 warmup 时，正式测量包含 CUDA 运行时初始化、kernel 加载、内部缓存建立、显存分配和 GPU 升频等一次性成本，因此均值偏高且方差很大。

一次 warmup 已经能够稳定 Forward 和完整训练步，但 Forward + Backward 仍出现明显波动。两次 warmup 后，三种模式的标准差均显著下降，结果与五次 warmup 基本一致。

本实验中两次 warmup 已经足以获得稳定结果，但五次 warmup 可以作为更稳妥的默认值。

备注

Warmup 不是为了让 GPU “知道马上要工作”，而是把首次执行产生的一次性初始化成本移出正式计时区间。具体需要多少次 warmup 与模型、算子、硬件和软件环境有关，不能假定一个固定数值对所有实验都最优。

12.4 阶段性结论

- GPU Benchmark 必须处理 CUDA 异步执行，并在计时边界进行同步；
- 关闭 autograd 对纯推理的显存占用影响显著，但对计算密集型模型的延迟影响较小；
- 反向传播耗时约为前向传播的两倍，完整训练步还需要额外支付优化器更新成本；
- 未进行 warmup 的结果包含大量一次性开销，均值和标准差均不可信；

- 实验配置应在运行期间保持不变，并为不同实验使用独立 YAML 和 CSV 文件。

下一步将使用 Nsight Systems 对 Forward、Backward 和 AdamW 中的 CUDA kernel 进行细粒度分析，观察端到端耗时具体分布在哪些算子上。